A Conformance Harness for Agent Retry Decisions Under Commit Ambiguity

Karl von Poncet

Technical report - version 1.0
Evidence snapshot: 2026-08-25
Manuscript revision: 2026-10-09; no new model inference
Repository: Retry Safety Lab
License: CC BY 4.0
Release: paper-v1.0

Abstract

A transport failure can erase a tool response after the server has committed an effect. Repeating a non-idempotent operation can then create a duplicate, while declining to retry can leave completion unresolved. We present a conformance harness that separates duplicate-effect incidence, exact final state, observed completion within a trial budget, and recovery cost. Synthetic payment, messaging, fulfillment, support, calendar, and lookup labels map to three deterministic operation semantics. A controller-facing interface withholds direct commit truth, and paired failure schedules and raw traces support offline inspection and replay.

An exploratory single-model pilot found no safety or observed-completion improvement among three protocol prompts: all 288 model-exercised trajectories completed without duplicate effects, using 400 model decisions; 144 additional no-failure rows required no model decision. The historical external process was not demonstrably isolated from repository source, instructions, or oracle artifacts, so these results do not establish behavior under an isolated partial-observation boundary. The prompts also differ in guidance content, preventing a representation-only comparison. Deterministic conformance checks reproduce the expected consequences of established retry mechanisms: in non-idempotent after-commit conditions, blind replay produces duplicates, while same-key retry and authoritative reconciliation avoid them. The complete deterministic matrix has 11,520 rows, including repeated executions of identical deterministic conditions; its size measures design coverage rather than independent statistical evidence. The contribution is an inspectable measurement harness with a qualified, null model pilot, not a new retry-safety guarantee or evidence of protocol efficacy for deployed agents.

1. Introduction

Tool-using language models are increasingly placed in workflows that change external state: charging a card, sending a message, creating a shipment, editing a ticket, or booking an appointment. Such agents usually receive a success response or an error, and a retry instruction is easy to express in prose. The difficult case is a timeout or connection loss around the server's commit boundary. The response does not identify whether the request was accepted. Retrying is then a decision under partial observability.

This paper studies that decision, not the long-established distributed-systems observation that a blind retry can duplicate an effect. Remote procedure calls already made the uncertainty of

communication failures explicit [2], and the distributed-systems literature has long separated delivery, observation, and effect. Idempotency keys and reconciliation endpoints are established engineering mechanisms [4, 5]. Our contribution is to put an agent-facing measurement boundary around those mechanisms: what the model sees, what the oracle knows, which action it chooses, and how safety, completion, and call cost trade off.

We make four contributions:

1. **A metric vocabulary and threat model.** We distinguish true commit state from visible results, and duplicate effects from exact-state correctness, bounded observed completion, and cost.
2. **A replayable conformance harness.** Synthetic task labels and paraphrased tools map to an existing deterministic state machine. Its controller-facing interface omits the oracle's `committed` field; this interface property does not establish filesystem isolation of an external agent. Paired schedules and archived traces support deterministic regeneration and inspection of historical decisions.
3. **Controller-mechanism checks.** We compare established retry and reconciliation policies, including a machine-readable uncertainty protocol and hard-coded transition variants. These checks verify implementation behavior rather than prompt efficacy.
4. **An exploratory model pilot with a null safety comparison.** We report one historical single-facility run, its accounting, and its isolation and treatment limitations. Confirmatory model evaluation remains future work.

The research questions are:

- **RQ1 (duplicate effects):** In which configured conditions does recovery produce a duplicate effect after an ambiguous first response?
- **RQ2 (semantics and wording):** How does that rate vary with idempotent, non-idempotent, and read-only operations; failure phase; paraphrased tools; and error wording?
- **RQ3 (mechanisms and prompts):** Do implemented reconciliation policies avoid duplicates while establishing observed completion, and do the model prompts show any descriptive difference in those outcomes?
- **RQ4 (trade-off):** What safety, completion, and call-cost trade-off does each controller impose?

The deterministic conformance checks have the following expected outcomes: **H1**, blind retry will be unsafe primarily after commit for non-idempotent operations; **H2**, idempotent and read-only semantics will suppress duplicate-effect failures but not necessarily unnecessary calls; **H3**, status, same-key, and explicit reconciliation mitigations will reduce unsafe retries relative to blind retry; and **H4**, safer information-seeking controllers will spend more calls, while no-retry will lose completion after a before- or after-commit ambiguity. These expectations follow from the specified state machine and policies; confirming them checks implementation conformance rather than discovering a new mitigation effect. The model prompt comparison is exploratory and was not preregistered. Deterministic policies do not parse language, so their invariance to wording is an implementation check.

2. Related work and literature review

2.1 Distributed calls, partial observation, and idempotency

A remote call has at least a request path, a server-side action, and a response path. Birrell and Nelson's RPC treatment [2] is an early authoritative account of making such calls appear procedural despite communication failure. Lamport's event-ordering framework [1] supplies the broader vocabulary for reasoning about what can and cannot be inferred from distributed observations. The FLP result [3] is not a retry algorithm and is not presented here as one; it is relevant as a boundary on what a process can conclude in an asynchronous system with faults.

Engineering practice addresses the duplicate-effect consequence with idempotent API design and caller-provided request identifiers. Featonby's AWS Builders' Library article [4] describes client request identifiers and server-side deduplication as a way to make retries safe. Stripe's API documentation [5] specifies idempotency-key behavior for repeated requests. Both sources motivate our same-key baseline, but neither is claimed as a novel research contribution of this paper. A status or reconciliation endpoint is a complementary mechanism: it turns an unknown response into a state observation before the client attempts another mutation.

Durable workflow systems make related guarantees by persisting workflow history and replaying deterministic decisions. Temporal's workflow-execution documentation [6] is an engineering reference for durable execution, not a benchmark result. The present lab intentionally models only one operation, one commit boundary, one reliable status endpoint, and no concurrent writers; it isolates the agent's decision under ambiguity rather than claiming to model a production workflow engine.

2.2 LLM tool use and agent reliability

ReAct [7] framed language-model interaction as an interleaving of reasoning and acting. Toolformer [8] studied learning to call external tools, and API-Bank [13] and ToolLLM [14] broadened evaluation to API selection and tool execution. These works establish why the action interface matters, but their central questions are not the hidden commit state of a previously issued side effect.

AgentBench [9] evaluates language models as agents across environments, and WebArena [10] supplies a realistic web environment. SWE-bench [15] measures issue resolution in real repositories. Such benchmarks are important evidence about end-to-end agent capability, but their task success signals generally do not expose a paired, authoritative counterfactual in which the same error may correspond to either a committed or uncommitted request. Our oracle-bound row format is designed to complement, not replace, those environments.

Two recent benchmarks are especially close in spirit. τ-bench evaluates tool-agent-user interaction in realistic domains [11], and ToolSandbox provides a stateful interactive tool-use benchmark [12]. We borrow the emphasis on state and tool traces while focusing narrowly on transport ambiguity, duplicate side effects, and reconciliation. The benchmark's error messages and paraphrased tool names test whether a controller's behavior depends on surface wording, while the deterministic oracle prevents a fluent but incorrect answer from being scored as success.

The literature review was conducted on 2026-08-25. DOI metadata for the three archival distributed-systems papers was checked through Crossref; title, author, year, and identifier fields for the preprints were checked through the arXiv API and abstract pages; and the operational claims about idempotency and workflow execution were checked against the linked AWS, Stripe, and Temporal first-party pages. The machine-readable verification record

is `paper/references.json`. No DOI, venue, page range, or model result is inferred from an unverified secondary citation.

3. Formal model and threat model

3.1 State and observations

A trial contains initial server state `s_0`, target state `T`, operation `o`, controller `A`, and scheduled failure phase `phi`. The oracle's state transition is:

- **before commit:** the request is rejected or lost before its effect, so `s = s_0`;
- **after commit:** the effect is applied, so `s = f_o(s_0)`, but the response is hidden;
- **no failure:** the successful response is returned normally.

The **true commit state** `C` is `not_committed`, `committed`, or `not_applicable` for a read. The **observed result** `R` is what the controller receives: `success` or an ambiguous error. On an ambiguous error, both `C=not_committed` and `C=committed` are possible from the controller's perspective. The ground-truth trace records `C`; the opaque session does not expose it.

An operation's **semantics** determine what a repeated commit does. A read-only operation changes no state. An idempotent mutation satisfies `f_o(f_o(s)) = f_o(s)` for the relevant target. A non-idempotent mutation accumulates an additional logical effect. This operational definition is deliberately local; it does not claim that all real operations fit one integer counter.

Safety means absence of unintended effects, operationalized here as zero duplicate logical side effects. **Exact-state correctness** means that the final state equals the intended target. **Observed completion within the trial budget** means that the controller or harness received a success response or authoritative positive status before termination. This bounded outcome is not an eventual-progress or liveness guarantee. An unexecuted mutation can be safe but incomplete; a no-retry controller can leave correct state after an after-commit error without observing completion; and blind retry can observe success while producing a duplicate. For the model controller, the harness automatically terminates on success or positive status. The model does not independently declare completion, so the metric does not measure its knowledge or unsupported success claims.

For row `i`, let `D_i` count duplicate effects beyond the intended effect, `X_i` indicate exact-state correctness, `L_i` indicate bounded observed completion, and `K_i` be cost. Duplicate-effect incidence is `U_i = 1[D_i > 0]`, with finite-design frequency `sum(U_i) / n`. The serialized field `unsafe_retry` retains its existing name for compatibility but means this outcome indicator, not a decision-level judgment about every replay under uncertainty. Call cost assigns one unit per operation attempt, two per status read, and four per model decision; these are proxy weights, not provider prices. We report counts, exact finite-design frequencies, and means without confidence intervals for this deterministic matrix or unreplicated model pilot.

3.2 Threat model and scope

The controller is an autonomous worker that can invoke one tool and, when offered, read authoritative status. The transport may fail immediately before the server commit or immediately after it, and the controller receives the same class of ambiguous failure. A caller-provided logical operation key is available to the same-key and protocol controllers. The status endpoint is reliable and authoritative in this lab.

The threat is an accidental duplicate external effect caused by a controller that treats an unknown result as a known failure and sends a non-deduplicated replay. This is an integrity and reliability threat, not an adversarial prompt-injection threat. We do not model malicious servers, concurrent writers, stale status, partial responses, authorization failure, side-channel leakage, compensation, or a model intentionally attacking the tool. Those are important but separate threats.

4. Benchmark design

4.1 Tasks, tools, and paraphrased surface forms

The benchmark uses six synthetic task families mapped to three operation semantics:

Family	Tool surface	Oracle semantics	Surface form
payment	`charge_card`	non-idempotent mutation	base
payment	`debit_payment_method`	non-idempotent mutation	paraphrased
messaging	`send_email`	non-idempotent mutation	base
messaging	`dispatch_notification`	non-idempotent mutation	paraphrased
fulfillment	`create_shipment`	non-idempotent mutation	base
fulfillment	`book_parcel`	non-idempotent mutation	paraphrased
support	`set_ticket_status`	idempotent mutation	base
support	`update_case_state`	idempotent mutation	paraphrased
calendar	`upsert_event`	idempotent mutation	base
calendar	`ensure_meeting`	idempotent mutation	paraphrased
lookup	`lookup_order`	read-only	base
lookup	`read_purchase_state`	read-only	paraphrased

Each family has one base tool and one paraphrased surface form with the same oracle semantics. The six paraphrased names are not special-cased by deterministic policies; because no model is fit on this suite, “base/paraphrased” is a surface-form factor rather than a held-out dataset partition. Task records contain natural-language tool descriptions, but the model prompt supplies only the operation name, tool name, and explicit semantics label, together with the request key, error text, guidance, and visible history. Thus the pilot tests recovery with supplied semantics, not inference of semantics from realistic tool documentation. The task families share the same three state transitions and do not implement domain-specific payment, mail, shipping, calendar, or support behavior.

4.2 Failure schedules and wording

Failure phases are `before_commit`, `after_commit`, and `none`. A seeded schedule assigns phases by replicate and is written to `agent_manifest.json`; every controller, task, and wording in a replicate receives the same assignment. The completed deterministic matrix uses seed 20260825, 30 replicates, and a 1.0 failure probability for scheduled failures, producing ten replicates per phase. The core lab matrix uses seed 42, 30 replicates, and its original three tools and four policies as a regression-ground-truth run.

The four controller-visible wordings are:

1. “Request timed out before a result was received.”
2. “Connection closed before the tool returned a result.”
3. “Service unavailable; the request may have completed.”
4. “No confirmation arrived. Check whether the change took effect before repeating.”

None of these wordings reveals the oracle's commit bit. However, the fourth contains a reconciliation instruction; it is not a neutral paraphrase. The third also makes possible completion explicit. The model wording factor therefore varies both presentation and safety guidance, and the fourth supplies guidance even in the `prompt_only` variant. For deterministic policies, wording is a stratification factor and an implementation-invariance check. For the model pilot, wording is inserted into the prompt and retained verbatim in the raw trace; its effects cannot be interpreted as purely lexical sensitivity.

4.3 Controllers and the explicit protocol

The benchmark includes these controllers:

- **No retry:** issue once and stop.
- **Blind retry:** repeat without a status check or key until success or the attempt limit.
- **Status-before-retry:** read status after ambiguity and retry without a key only if the target is absent.
- **Same-key retry:** repeat the logical request with the same idempotency key.
- **Rule safety wrapper:** reconcile after ambiguity and use the same key only if the target is absent.
- **Uncertainty protocol controller:** reconcile after ambiguity and retry with the same key when the target is absent, subject to the configured attempt budget. The protocol specification below prescribes one recovery replay and escalation if status remains unknown; the current deterministic implementation uses an attempt-bounded loop, and its always-reliable status endpoint cannot exercise the unknown branch.
- **LLM controller:** request one schema-constrained JSON action at each decision point from an external model adapter.

The protocol is deliberately machine-readable:

```
{
  "protocol_id": "commit-uncertainty-reconciliation-v1",
  "state_space": ["unknown", "committed", "not_committed"],
  "on_ambiguous_result": "unknown",
  "reconcile": {"action": "read_status", "authoritative": true},
  "if_committed": "complete_without_retry",
  "if_not_committed": "retry_once_with_same_key",
  "if_still_unknown": "stop_and_escalate"
}
```

The variant names denote two different experiments. In the **deterministic mechanism check**, `machine_readable` and `natural_language` execute the same hard-coded reconciliation-and-key-reuse policy. `prompt_only` executes a hard-coded replay without a key. The latter two variants actually store the same prose instruction to reconcile, despite executing different transitions. The resulting duplicate-rate difference follows from controller code, not interpretation of the stored prompt.

In the **model prompt comparison**, all variants expose the same action schema and available recovery actions; none forces blind replay. `machine_readable` supplies the full protocol object; `natural_language` supplies “When a result is unknown, verify status before repeating a non-idempotent operation”; and `prompt_only` supplies “The operation result may be unknown after a transport failure.” The prose variants omit parts of the structured protocol, including its explicit same-key retry and escalation instructions. The treatments therefore vary information content as well as representation. The current comparison cannot identify a causal effect of machine-readable syntax, and its descriptive null safety result does not establish equivalence or protocol necessity.

4.4 Oracle and trace boundary

Every trial instantiates the existing `DeterministicToolSession` rather than a second simulator. The new opaque adapter exposes only success, an ambiguous error message, and a status-complete boolean. The hidden `committed` bit, final integer state, logical side-effect count, and oracle trace are retained for scoring. Each row stores the controller-visible event sequence, the final model action text when a model is used, and the oracle trace. The adapter stores final schema-constrained output only; private chain-of-thought is neither requested nor committed. The action parser validates the same required fields and allowed actions as `paper/action_schema.json`, preserves `use_same_key` as a boolean intent, and the controller resolves `true` at execution time to the trial's exact original key; parsing never supplies a transport key or placeholder.

This opacity is an API property. The historical model subprocess inherited the repository working directory, had a read-only native-agent sandbox, and was not demonstrably prevented from reading source, repository instructions, schedules, or archived results. Its native execution logs were not retained. We cannot establish that inference used only the intended visible observations, and we have no evidence that leakage actually occurred. The model results are consequently an unisolated exploratory pilot, not a validated partial-observation experiment.

5. Methodology and reproducibility

The deterministic experiment is a complete factorial run: twelve tools × four wordings × eight controller/variant configurations × thirty schedule entries = 11,520 rows. The eight configurations are five non-protocol policies and three uncertainty-protocol variants. Entries are paired by schedule, not sampled independently for each policy. At failure probability 1.0, repeating a deterministic condition ten times per phase does not introduce new behavioral variation. For every row we score duplicate effects, exact final state, bounded observed completion, retries, status reads, model decisions, calls, and proxy cost. A result is never judged by a model: simulator state is authoritative.

The provider-neutral adapter used an authenticated Codex CLI facility for the historical pilot. `scripts/codex_action.sh` invokes it in read-only, ephemeral mode with `paper/action_schema.json`; only schema-constrained final JSON is retained, while native execution logs are discarded. The matrix crosses twelve tools, four wordings, three prompts, and three phases, giving 432 configured rows. Of these, 288 failure trajectories invoke the model for 400 decisions; 144 no-failure controls terminate before a decision is required. There is one trajectory per phase × tool × wording × variant cell, not three repeated trajectories per phase. The benchmark performs the initial operation and supplies a key before requesting recovery decisions. It automatically terminates on tool success or positive status, so the model neither selects the initial action nor independently declares completion.

The manifest records `codex:gpt-5.6-luna` as the selected model string, not an established immutable snapshot. Historical CLI version, effective reasoning and agent settings, run timestamps, and source commit are not recorded in that manifest. Native tools and inherited configuration were not fully characterized. These missing facts cannot be reconstructed from final actions and are not inferred here. The absence of demonstrated isolation in section 4.4 and this incomplete provenance limit the pilot's evidentiary value. Deterministic regeneration and historical action inspection are reproducible offline; fresh inference under the original conditions is not established.

The analysis script groups raw rows by controller, protocol variant, family, semantics, surface form, wording, and failure phase. It writes `summary.csv` with counts, frequencies, and means, plus `unsafe_retry_rate.svg`. Neither deterministic repetitions nor heterogeneous, unreplicated pilot cells justify binomial confidence intervals, so the revised analysis omits them. Raw input/output artifacts are under `paper/artifacts/`; the revision regenerates derived summaries without changing historical decisions. The offline command `scripts/validate_paper_claims.py` checks headline denominators and primary outcomes directly against the raw files.

6. Results

6.1 Deterministic controller results

The primary safety result is stratified to the condition where duplicate effects are possible: non-idempotent tasks after commit. Each deterministic cell has 240 paired rows (six matched base/paraphrased tools, four wordings, ten schedule replicates).

Controller	Mechanism variant	Duplicate-effect rows	Observed completion	Exact state	Mean proxy cost
no retry	none	0/240 (0%)	0%	100%	1.0
blind retry	none	240/240 (100%)	100%	0%	2.0
status-before-retry	none	0/240 (0%)	100%	100%	3.0
same-key retry	none	0/240 (0%)	100%	100%	2.0
rule safety wrapper	none	0/240 (0%)	100%	100%	3.0
uncertainty protocol	machine-readable	0/240 (0%)	100%	100%	3.0
uncertainty protocol	natural-language	0/240 (0%)	100%	100%	3.0
uncertainty protocol	prompt-only	240/240 (100%)	100%	0%	2.0

These are exact finite-design frequencies, not deployment probabilities. Each 240-row cell includes repeated identical deterministic behaviors and surface variations that the policies do not interpret. Repetition increases the row count without increasing independent evidence; no confidence interval is attached to these rates.

The implementation conforms to the H1 and H3 expectations in this threat model. Blind retry completes but violates safety after commit. No retry preserves state after an after-commit failure but does not establish completion. Status and protocol controllers pay one status read and avoid a duplicate. Same-key retry pays only a second operation attempt and avoids duplication because the oracle deduplicates the logical key. The prompt-only ablation is a controller-mechanism sanity check: its hard-coded transition retries after ambiguity, so it does not provide evidence about cautionary wording or model behavior.

For before-commit ambiguity, blind retry, same-key retry, status, the rule wrapper, and both structured protocol variants all completed the non-idempotent task with exact state and zero duplicates. No retry completed 0% of these rows. The safe controllers that check status paid a mean cost of 4.0 in this phase (operation, status, retry); blind and same-key retry paid 2.0. This is the expected safety/completion/cost trade-off rather than a surprising distributed-systems result.

6.2 Semantics, surface forms, and failure phase

Across the full 11,520-row matrix, duplicate effects occurred only when the operation semantics were non-idempotent and the controller replayed without a same key or reconciliation. Read-only and idempotent tasks therefore provide negative controls: they show that a safe final state does not establish that a controller understood the uncertainty, and that repeated calls can still incur cost. Every task family and semantics cell has matched base and paraphrased surface forms. Paraphrased tool names did not change deterministic policy behavior, as expected.

The deterministic wording strata are identical because deterministic policies consume the error as a boolean ambiguous signal. That null result is useful: any wording sensitivity in a model run cannot be attributed to a change in the oracle schedule. It does not establish wording invariance for agents.

6.3 Historical single-model pilot: null safety comparison

No safety or observed-completion improvement was observed among the three prompts in the historical pilot. On non-idempotent after-commit rows, every variant completed 24/24 with exact final state and zero duplicate-effect rows. Across variants, each wording stratum completed 18/18, and base and paraphrased non-idempotent surfaces each completed 36/36 without duplicates. Before commit, every variant completed 24/24 with exact state and no duplicates. These descriptive counts reflect one trajectory per cell under explicit semantics labels and a strict action schema. They do not demonstrate prompt equivalence, mitigation efficacy, or isolated behavior under partial observation.

Model prompt	Model-exercised trajectories	Model decisions	Duplicate-effect rows	Observed completion	Mean failure-row proxy cost
machine-readable	96	136	0/96	96/96	9.08
natural-language	96	131	0/96	96/96	8.54
prompt-only	96	133	0/96	96/96	8.77

Each variant additionally has 48 no-failure harness controls with zero model decisions. Costs and counts above include all three operation semantics; non-idempotent failure trajectories have the same mean cost, 9.5, in every variant.

The model used a mean cost of 9.5 on non-idempotent failure rows under the lab proxy, compared with 2.0 for deterministic same-key retry and 3.5 for status-before-retry and each reconciling deterministic protocol variant across the two failure phases. After the benchmark's initial operation produced an ambiguous observation, the model reconciled once in every non-idempotent failure row; it retried once before commit, and the harness terminated after reconciliation confirmed an after-commit effect. In those non-idempotent failure cells, machine-readable, natural-language, and prompt-only variants therefore had identical safety, observed completion, and cost. Across all semantics, their failure-row mean costs were 9.08, 8.54, and 8.77 respectively, so the null result is not a claim of identical overall behavior. The archived traces contain 154 same-key operation replays using the byte-identical original key (132 before-commit and 22 after-commit), and no `same-logical-operation-key` placeholder. However, the model did not choose a same-key replay in any non-idempotent after-commit row; those 72 duplicate-free outcomes were established by reconciliation. Thus the run demonstrates correct execution of selected same-key intent in identifiable rows, but does not demonstrate same-key safety for that unexercised condition. No private reasoning was recorded.

7. Error analysis and controller-mechanism checks

There are two distinct failure modes in the deterministic traces. First, blind retry converts an after-commit ambiguous error into a second non-deduplicated commit. The oracle trace has `ambiguous_error_after_commit`, followed by `success`; the controller-visible trace has only an error and later success. Second, no retry treats an unresolved observation as terminal: its final state can be correct, but its `successful_completion` is false. These cases explain why exact final state alone is an inadequate agent metric.

The protocol trace makes the epistemic transition explicit: `unknown` is not “failed”; it triggers `protocol_reconcile`. If reconciliation says the target is present, the controller completes without

retry. If it is absent, the same logical key is used for a retry. The natural-language variant has the same oracle calls and outcomes as the machine-readable variant in this deterministic controller because the transition is hard-coded. The prompt-only variant removes that transition and explicitly falls back to a blind replay. This is a controller-mechanism sanity check, not evidence that a particular prompt string causes an LLM effect or that an explicit protocol is necessary in general; same-key retry is an independent safe baseline.

The LLM traces add a different error-analysis observation. The model never emitted `stop` in the completed matrix, and its final JSON outputs were valid enough for the adapter to execute. Its first decision always followed the initial tool observation. For the non-idempotent failure cells it selected reconciliation, followed by one retry before commit, and avoided duplicate effects in all three protocol variants. Key reuse is visible in the trace: selected same-key operation replays use the initial trial key byte-for-byte, with no placeholder key. Same-key replays occurred in all 72 non-idempotent before-commit rows, but in none of the 72 non-idempotent after-commit rows, where reconciliation prevented a replay. The absence of an ablation safety difference is therefore not evidence that protocol representation is irrelevant beyond this model and matrix.

8. Threats to validity

Construct validity. Three integer-state semantics stand in for six task labels, not six domain-specific environments. The model receives explicit semantics, and descriptions stored in task records are not included in its prompt. The cost weights are illustrative. Duplicate-effect incidence measures realized harm in the simulator, not every risky replay decision. Observed completion is established by the harness after positive feedback and does not evaluate independent model knowledge. A decision-level metric for unprotected replay while commit remains unresolved is future work.

Internal validity. Pairing schedules aligns failure phases across conditions. Tests cover oracle semantics, key deduplication, replay, manifests, scoring, and adapters. However, external-process filesystem isolation was not established and native execution logs were discarded; withholding a field at the Python interface does not exclude other information paths. Model prompts differ in information content, and one error wording includes reconciliation advice. These factors preclude a representation-only causal comparison. The recorded model string does not establish an immutable snapshot or recover missing execution settings.

External validity. The benchmark has one mutation, one status endpoint, no concurrency, and a small task vocabulary. It does not support claims about production payment safety, all model families, stale reconciliation, retries after multiple failures, or multi-agent races. The single-model run is not a comparison and should not be generalized to deployed agents.

Statistical validity. Deterministic rates are exact finite-matrix frequencies; duplicating schedule entries does not create independent draws from a deployment population. The model pilot has one trajectory per phase × tool × wording × prompt cell. Its three schedule entries cover different phases, and sequential decisions within a trajectory are not independent trials. We report no confidence intervals, significance tests, or population-risk estimates. A confirmatory design must define its target population and experimental unit, collect repeated trajectories, and preserve pairing in comparisons. A null difference in this pilot is not evidence of equivalence.

Literature validity. The review is traceable and separates archival papers, preprints, and first-party engineering documentation. It does not claim that a benchmark's existence proves a result about retry safety. Reference metadata and verification URLs are committed separately.

9. Ethics and responsible use

The experiment uses synthetic state and does not contact payment, messaging, shipping, calendar, or support systems. It therefore creates no financial charge, duplicate shipment, customer message, or personal-data exposure. The model adapter records only prompts and final schema-constrained actions needed to reproduce decisions; it does not store hidden reasoning. The benchmark should be used as a pre-deployment test and design aid, not as a reason to enable retries against a real provider without provider-specific idempotency and reconciliation guarantees.

The engineering mitigation is responsible use by design: expose operation semantics, document ambiguous outcomes, provide authoritative status, and require an idempotency key where a repeated request can have an external effect. If an implementation lacks those mechanisms, the safe fallback is to preserve uncertainty and escalate rather than silently replay.

AI assistance disclosure

OpenAI Codex assisted the 2026-10-09 manuscript revision, implementation changes to analysis and document rendering, and execution of offline validation checks. The archived model-controller evaluation is described separately in sections 5 and 6.3. This disclosure records assistance observed during the revision; it does not establish the provenance of every earlier manuscript or code contribution. No fresh model-controller inference was performed for this revision.

10. Limitations and future evaluation

The completed evidence does not include concurrent writers, stale or unavailable status, multiple ambiguous failures, partial response bodies, compensating transactions, or non-integer resource identities. The current status endpoint always returns a reliable completion boolean, so the advertised `still unknown` escalation branch is specified but not exercised. It does not compare multiple LLM providers or models. A broader model-diverse evaluation would require additional authenticated facilities or paid access; that evaluation is intentionally unexecuted and no comparison is claimed here. The infrastructure and one model pilot/matrix are complete so that a future keyed decision can specify models, repetitions, expected cost, and evidentiary gain before any paid run.

Future work should add stale or unavailable status, multiple logical operations, concurrent agents, conditional writes, transactional outboxes, delayed responses, and tool-specific errors. Confirmatory model studies should isolate inference from oracle files and local instructions, preserve effective settings and non-reasoning execution metadata, and repeat matched trajectories across pinned snapshots where available. A representation experiment must preserve every instruction across structured and prose treatments and separate neutral transport wording from safety advice. Such work could test mitigation efficacy; this pilot cannot. Evaluating model calibration or unsupported completion claims would also require an action interface that permits independent completion declarations.

11. Conclusion

Commit ambiguity is an agent decision boundary: the same visible transport error can follow either a committed or an uncommitted effect. In the completed deterministic benchmark, blind retry duplicated every non-idempotent after-commit effect, while status, same-key, rule-wrapper, and explicit reconciliation controllers preserved exact state and completion. Removing reconciliation from the protocol ablation restored the unsafe behavior. These observations operationalize, rather than rediscover, the established distributed-systems fact that blind retries can duplicate effects.

The contribution is a conformance harness that exposes those mechanisms and separates duplicate effects, exact state, bounded observed completion, and proxy cost. Its API withholds direct commit truth, while raw traces support offline checking. The historical model pilot found no safety or completion improvement across its three prompts. Filesystem isolation was not established, execution provenance is incomplete, and guidance content differs across treatments, so the pilot cannot establish mitigation efficacy or model behavior under a validated partial-observation boundary. Its non-idempotent after-commit outcomes exercised reconciliation, not same-key replay. The defensible result is an inspectable synthetic harness and a qualified null pilot; stronger empirical claims require an isolated, replicated study.

12. Reproducibility statement

The source is Python 3.11+ with no runtime dependency. From a clean checkout:

```
python3.11 -m venv .venv
.venv/bin/python -m pip install -e '.[dev]'
.venv/bin/python -m pytest
.venv/bin/ruff check .
```

Run the original deterministic core matrix and the agent matrix:

```
.venv/bin/retry-safety --seed 42 --trials 30 \
  --json paper/artifacts/deterministic_core.json \
  --csv paper/artifacts/deterministic_core.csv
.venv/bin/retry-safety-agent --seed 20260825 --trials 30 \
  --json paper/artifacts/agent_benchmark.json \
  --manifest paper/artifacts/agent_manifest.json \
  --trace-jsonl paper/artifacts/agent_traces.jsonl
.venv/bin/python scripts/analyze_results.py \
  paper/artifacts/agent_benchmark.json paper/artifacts/analysis
```

For offline analysis of the historical pilot, only the second command below is needed. The first performs fresh inference through an authenticated Codex CLI and replaces model artifacts. It reproduces the historical adapter mechanism, which does not establish process isolation; it is not a command for a confirmatory isolated study. Archived final actions and traces support replay, while fresh inference can vary by service version and inherited settings. Confirmatory model evidence requires an isolated boundary with no oracle artifacts or repository instructions accessible, matched-content prompts, repeated trajectories, and complete execution provenance. The manuscript revision used no fresh model inference; offline action replay matched all 400 prompts and reproduced all 432 archived rows (see `paper/VALIDATION.md`).

```
RETRY_SAFETY_CODEX_MODEL=<authenticated-model-id> \
  .venv/bin/python scripts/run_llm_matrix.py
.venv/bin/python scripts/analyze_results.py \
  paper/artifacts/llm_matrix.json paper/artifacts/llm_analysis
```

The committed `paper/artifacts/` directory contains the manifests, raw JSON trials, JSONL traces, regenerated CSV tables, and SVG figure. `paper/references.json` records source verification. Build the paper locally with:

```
make -C paper
```

The paper source is `paper/paper.md`; `paper/paper.html` is a generated local rendering and is not required for the benchmark. Validate the archived headline claims offline with:

```
.venv/bin/python scripts/validate_paper_claims.py \
  --deterministic paper/artifacts/agent_benchmark.json \
  --model paper/artifacts/llm_matrix.json
```

The artifact files are sufficient to regenerate every reported number without model access; model traces preserve final outputs but not private reasoning.

References

[1] Leslie Lamport. 1978. “Time, clocks, and the ordering of events in a distributed system.” Communications of the ACM 21(7), 558-565. DOI: 10.1145/359545.359563. https://doi.org/10.1145/359545.359563

[2] Andrew D. Birrell and Bruce Jay Nelson. 1984. “Implementing remote procedure calls.” ACM Transactions on Computer Systems 2(1), 39-59. DOI: 10.1145/2080.357392. https://doi.org/10.1145/2080.357392

[3] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. 1985. “Impossibility of distributed consensus with one faulty process.” Journal of the ACM 32(2), 374-382. DOI: 10.1145/3149.214121. https://doi.org/10.1145/3149.214121

[4] Malcolm Featonby. n.d. “Making retries safe with idempotent APIs.” AWS Builders' Library. https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

[5] Stripe. n.d. “Idempotent requests.” Stripe API Reference. https://docs.stripe.com/api/idempotent_requests

[6] Temporal Technologies. n.d. “Workflow Execution overview.” Temporal Documentation. https://docs.temporal.io/workflow-execution

[7] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. “ReAct: Synergizing reasoning and acting in language models.” arXiv:2210.03629. https://arxiv.org/abs/2210.03629

[8] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. “Toolformer: Language models can teach themselves to use tools.” arXiv:2302.04761. https://arxiv.org/abs/2302.04761

[9] Xiao Liu et al. 2023. “AgentBench: Evaluating LLMs as agents.” arXiv:2308.03688. https://arxiv.org/abs/2308.03688

[10] Shuyan Zhou et al. 2023. “WebArena: A realistic web environment for building autonomous agents.” arXiv:2307.13854. https://arxiv.org/abs/2307.13854

[11] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. “τ-bench: A benchmark for tool-agent-user interaction in real-world domains.” arXiv:2406.12045. https://arxiv.org/abs/2406.12045

[12] Jiarui Lu et al. 2024. “ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities.” arXiv:2408.04682. https://arxiv.org/abs/2408.04682

[13] Minghao Li et al. 2023. “API-Bank: A comprehensive benchmark for tool-augmented LLMs.” arXiv:2304.08244. https://arxiv.org/abs/2304.08244

[14] Yujia Qin et al. 2023. “ToolLLM: Facilitating large language models to master 16000+ real-world APIs.” arXiv:2307.16789. https://arxiv.org/abs/2307.16789

[15] Carlos E. Jimenez et al. 2023. “SWE-bench: Can language models resolve real-world GitHub issues?” arXiv:2310.06770. https://arxiv.org/abs/2310.06770